

Praktikum 5 — Feature Engineering and Dimensionality Reduction

Data Science • Magister Ilmu Komputer, FMIPA UGM Companion to **Chapter 5**. Assignment weight: **3%**.

What you will do

1. Encode categorical variables several ways, and break one of them on purpose.
2. See which models need scaling, which scaler to pick, and how to let a straight line bend.
3. Build RFM features from a transaction log.
4. Select features by filter, wrapper and embedded methods.
5. Derive PCA numerically and read its loadings.
6. **Measure the cost of data leakage** — the most important cell in this course.

The datasets. `data/transactions.csv` (30,000 transactions) and `data/credit.csv` (4,000 loan applications with a default flag).

How to read this notebook

Every code cell is followed by a short explanation in a box like this one:

What just happened. What the code did, in plain words.

How to read the output. Which numbers to look at, and what they mean.

Why it matters. The one idea to take away.

Run a cell, look at its output, *then* read the box underneath. If the explanation and your output disagree, re-run the notebook from the top.

Three ideas are used in almost every cell, so learn them once here.

- **ROC-AUC** scores how well a model *ranks* cases: 1.0 means every positive case is ranked above every negative one; **0.5 means no better than a coin flip**. Below about 0.6 is weak, 0.8 is decent, above 0.9 is strong.
- **Cross-validation** (`cross_val_score`) splits the data into 5 parts, trains on 4 and scores on the 5th, five times, and averages. It estimates how the model does on data it has *not* seen.
- `fit` **versus** `transform` . `fit` *learns* something from data (a mean, a list of categories, a set of chosen columns); `transform` *applies* it. Every leakage bug in §6 comes from fitting on rows you later use for scoring.

```

import sys
from pathlib import Path

ROOT = Path.cwd()
while not (ROOT / "checks.py").exists() and ROOT != ROOT.parent:
    ROOT = ROOT.parent
sys.path.insert(0, str(ROOT))

import checks
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

DATA = ROOT / "data"
plt.rcParams.update({"figure.dpi": 110, "axes.spines.top": False,
                    "axes.spines.right": False, "axes.grid": True,
                    "grid.alpha": 0.3, "axes.titleweight": "bold"})
pd.set_option("display.width", 110)

```

0. Why representation is the problem (DEMO)

Every model class has an **inductive bias** — a set of relationships it can express easily. A linear model draws straight lines; a decision tree cuts the space into boxes with splits parallel to the axes. Feature engineering is re-expressing the data so the relationship you care about is one the model can see.

Here the truth is that the chance of default rises with `debt / income`: around a ratio of 1.2 it climbs from unlikely to likely. We give two models either the raw columns, or the raw columns plus that one ratio.

```

from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import cross_val_score, StratifiedKFold
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

rng = np.random.default_rng(0)
n = 2000
income = rng.lognormal(16, 1.0, n)
debt = income * rng.beta(2, 3, n) * 3
p_risk = 1 / (1 + np.exp(-8 * (debt / income - 1.2))) # the truth is a RATIO
risk = (rng.random(n) < p_risk).astype(int)

cv = StratifiedKFold(5, shuffle=True, random_state=0)
raw = pd.DataFrame({"debt": debt, "income": income})
eng = raw.assign(dti=debt / income)

models = {"logistic regression": make_pipeline(StandardScaler(), LogisticRegression()),
          "decision tree (depth 3)": DecisionTreeClassifier(max_depth=3, random_state=0)}

for model_name, model in models.items():
    for name, X in [("debt + income (raw)", raw), ("+ debt/income ratio", eng)]:
        s = cross_val_score(model, X, risk, cv=cv, scoring="roc_auc")
        print(f"{model_name:<24} {name:<22} ROC-AUC = {s.mean():.3f}")
    print()

```

What just happened. We simulated 2,000 borrowers whose risk depends only on the ratio debt/income, then scored two models with and without a `dti` column. Nothing else changed — same rows, same model, same folds.

How to read the output. The **tree** is the dramatic one: about 0.80 on the raw columns, about 0.96 once it is given the ratio. The **logistic regression** gains less (about 0.95 to 0.96).

Why the two models react so differently. "Debt/income above 1.2" is the region above the line `debt = 1.2 × income` — a *diagonal* line through the origin. A tree can only cut horizontally or vertically, so it has to build a staircase of many small boxes to follow a diagonal, and a depth-3 tree runs out of steps. A logistic regression draws straight lines of any angle, so it already approximates this boundary — but only approximately, because incomes here range over two orders of magnitude and a single straight line cannot rank a rich and a poor borrower on the same ratio correctly.

Why it matters. Given `dti`, both models see the whole problem in a single column. No amount of hyperparameter tuning on the raw columns would do what that one division did.

1. Encoding categorical variables (DEMO)

Models work with numbers. A column like `purpose = "pendidikan"` must be turned into numbers first, and *how* you do it decides what the model believes about the categories.

```
credit = pd.read_csv(DATA / "credit.csv")
print(credit.shape)
print(credit.dtypes.to_string())
credit.head(3)
```

What just happened. We loaded 4,000 loan applications with 14 columns.

How to read the output. `int64` / `float64` columns are numeric; the five `object` columns are text (categories): `education`, `purpose`, `housing`, `employment_type` and the target `default` (yes / no). Money columns are in rupiah, so they run into the millions — keep that in mind for §2.

```
print("Cardinality of each categorical column:")
for c in credit.select_dtypes("object").columns:
    print(f" {c:<18} {credit[c].nunique():>3} levels  "
          f"{sorted(credit[c].dropna().unique())}")
```

What just happened. For each text column we counted its distinct values (its **cardinality**) and listed them.

How to read the output. Every column has only 3–5 levels. That matters: the right encoding depends on how many levels there are. A handful of levels → one-hot is fine. Thousands of levels → you need target encoding or hashing (end of this section).

Notice that `education` has a natural order (`SMA < D3 < S1 < S2`) while `purpose` does not. That difference decides between ordinal and one-hot.

One-hot: the default for nominal variables

```
from sklearn.preprocessing import OneHotEncoder

enc = OneHotEncoder(handle_unknown="infrequent_if_exist",
                    min_frequency=0.01, sparse_output=False)
oh = enc.fit_transform(credit[["purpose", "housing"]].fillna("Unknown"))
print("shape:", oh.shape)
print("features:", list(enc.get_feature_names_out()))
```

What just happened. One-hot encoding made one 0/1 column per level. A loan for education gets `purpose_pendidikan = 1` and 0 in the other four purpose columns.

How to read the output. 2 text columns became 8 numeric ones (5 purposes + 3 housing types). No level is "bigger" than another — each is just present or absent, which is exactly right for a variable with no order.

Three parameters carry the practical weight:

- `drop="first"` — needed **only for OLS**, where the dummies otherwise sum to a column of ones and make $X^T X$ singular. Dropping hurts tree feature importances, so keep it for everything else.
- `handle_unknown` — a level *will* appear at prediction time that was absent at training time. Without this, `transform` raises and your service 500s.
- `min_frequency` — pools rare levels, preventing a column that is 1-in-20,000 and pure noise.

Ordinal: only for genuinely ordered variables

```
from sklearn.preprocessing import OrdinalEncoder

# WRONG: let the encoder sort alphabetically
wrong = OrdinalEncoder().fit(credit[["education"]].fillna("SMA"))
print("alphabetical order imposed:", list(wrong.categories_[0]))

# RIGHT: state the order explicitly
order = ["SMA", "D3", "S1", "S2"]
right = OrdinalEncoder(categories=order, handle_unknown="use_encoded_value",
                      unknown_value=-1).fit(credit[["education"]].fillna("SMA"))
print("explicit order      :", list(right.categories_[0]))

print("\nAlphabetically, D3 < S1 < S2 < SMA -- which says a high-school")
print("graduate outranks a master's. Always pass `categories`.")
```

What just happened. Ordinal encoding replaces each level with one number: its position in a list (0, 1, 2, 3). We built the list twice — once letting the encoder choose, once stating it ourselves.

How to read the output. Left alone, the encoder sorts *alphabetically*, so SMA (high school) gets code 3, the highest. With `categories=` given, SMA is 0 and S2 is 3, which matches reality.

Why it matters. The model cannot tell a meaningful order from an accidental one — it just sees bigger and smaller numbers. And never ordinal-encode a variable with *no* order (like cities): the model will believe city 2 is "twice" city 1.

Target encoding: powerful, and the easiest way to leak

Target encoding replaces each level with the **average of the target** for that level — e.g. each city with its default rate. One column, however many levels there are.

The naive version, $\text{enc}(k) = \bar{y}_k$, **memorises**: a level appearing once is encoded with that row's own target value.

```
demo = pd.DataFrame({
    "kota": ["Yogya", "Yogya", "Solo", "Solo", "Klaten"], # Klaten appears once
    "y":    [1, 0, 1, 1, 1],
})
naive_map = demo.groupby("kota")["y"].mean()
print("naive target encoding:")
print(naive_map.to_string())
print("\nKlaten -> 1.0, which IS that row's own label. A model reading this")
print("column achieves perfect training accuracy and learns nothing.\n")

# Smoothing shrinks small groups towards the global mean.
def smooth_encode(counts, means, global_mean, m=20):
    return (counts * means + m * global_mean) / (counts + m)

g = demo.groupby("kota")["y"].agg(["count", "mean"])
g["smoothed"] = smooth_encode(g["count"], g["mean"], demo["y"].mean(), m=20)
print("with smoothing (m=20):")
print(g.round(4).to_string())
print("\nKlaten now sits near the global mean: one observation is almost no evidence.")
```

What just happened. Five toy rows, three cities. We encoded each city by its mean y , first naively, then with **smoothing**.

How to read the output. Naively, Klaten = 1.0 — but Klaten has one row, so "1.0" is simply that row's own answer copied into a feature. That is cheating: the feature *contains the label*.

Smoothing mixes each city's own mean with the overall mean (0.8 here), weighted by how much evidence the city has: $\frac{n_k \bar{y}_k + m \cdot \bar{y}}{n_k + m}$. With one row and $m = 20$, the city's own data gets weight $1/21$, so Klaten drops to about 0.81 — close to the overall mean, as one observation deserves.

```
# scikit-learn's TargetEncoder does smoothing AND cross-fitting by default.
from sklearn.preprocessing import TargetEncoder

y = (credit["default"] == "yes").astype(int)
te = TargetEncoder(smooth="auto", cv=5, random_state=0)
te_out = te.fit_transform(credit[["purpose"]].fillna("Unknown"), y)
print("target-encoded 'purpose', first 5:", np.round(te_out[:5, 0], 4))
print("global default rate          :", round(y.mean(), 4))
```

What just happened. The real thing, on the credit data. **TargetEncoder** smooths *and* **cross-fits**: it splits the rows into 5 folds and encodes each row using only the *other* 4 folds, so no row ever sees its own label.

How to read the output. Each loan purpose becomes a number near the overall default rate (about 0.20): purposes with more defaults sit a little above it, safer ones a little below. That spread is the information the column carries.

Why it matters. Smoothing reduces the cheating; cross-fitting removes it. Use **TargetEncoder** rather than a hand-written `groupby().mean()`.

Frequency and hashing: when there are too many levels (short demo)

Our credit columns have 3–5 levels. Real data often has hundreds or thousands (postcodes, product IDs). We simulate a postcode column with 800 possible values to see how two cheaper encodings behave.

```

from sklearn.feature_extraction import FeatureHasher

rng_k = np.random.default_rng(3)
weights = 1 / np.arange(1, 801) ** 1.1          # a few common, many rare
codes = np.array([f"KP{i:03d}" for i in range(800)])
kode_pos = pd.Series(rng_k.choice(codes, size=20_000, p=weights / weights.sum()))

# Frequency encoding: each level -> its share of the rows (1 column)
freq = kode_pos.map(kode_pos.value_counts(normalize=True))

# Hashing: each level -> one of 64 buckets chosen by a hash function (64 columns)
hasher = FeatureHasher(n_features=64, input_type="string")
H = hasher.transform([[k] for k in kode_pos])

levels = kode_pos.unique()
bucket_of_level = hasher.transform([[k] for k in levels]).toarray().col
per_bucket = pd.Series(bucket_of_level).value_counts()

print(f"distinct postcodes seen      : {kode_pos.nunique()}")
print(f"one-hot would need          : {kode_pos.nunique()} columns")
print(f"frequency encoding needs     : 1 column (e.g. {kode_pos[0]} -> {freq[0]:.4f})")
print(f"hashing needs                 : {H.shape[1]} columns")
print(f"postcodes sharing one bucket : {per_bucket.mean():.1f} on average, "
      f"up to {per_bucket.max()}")

```

What just happened. Two ways to encode a column with hundreds of levels without creating hundreds of columns.

How to read the output. One-hot would need one column per postcode seen. **Frequency** encoding uses a single column: "how common is this postcode?" **Hashing** squeezes every postcode into 64 buckets; the price is that several unrelated postcodes land in the *same* bucket (a **collision**), and the model cannot tell them apart.

Why it matters. Frequency encoding helps only if popularity itself predicts the target. Hashing has a fixed width and handles never-seen levels automatically, which suits streaming data — but a bucket has no meaning a human can read. Neither is free; you choose which loss to accept.

2. Scaling: who needs it (DEMO)

Scaling matters when the model is **distance- or variance-based**, or **penalised**. It is irrelevant to trees, whose splits depend only on the order of values.

We use four columns: `age` (tens), `income` and `debt` (tens of millions of rupiah), and the ratio `pti` = monthly payment / monthly income (around 1), which is the most predictive of the four.

```

from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC

X_demo = credit[["age", "income", "debt", "monthly_payment"]]
X_demo = X_demo.fillna(X_demo.median())
X_demo = X_demo.assign(
    pti=X_demo["monthly_payment"] / (X_demo["income"] / 12).clip(lower=1)
)[["age", "income", "debt", "pti"]]
print("Feature ranges -- income is ~7 orders of magnitude larger than pti:")
print(X_demo.describe().T[["min", "max"]].round(2).to_string())

print("\nEffect of scaling on cross-validated ROC-AUC:\n")
for name, model in [("k-NN", KNeighborsClassifier(15)),
                    ("SVM (RBF)", SVC(probability=False)),
                    ("Random forest", RandomForestClassifier(
                        n_estimators=60, random_state=0))]:
    unscaled = cross_val_score(model, X_demo, y, cv=cv, scoring="roc_auc").mean()
    scaled = cross_val_score(make_pipeline(StandardScaler(), model),
                             X_demo, y, cv=cv, scoring="roc_auc").mean()
    print(f" {name:<16} unscaled {unscaled:.4f}   scaled {scaled:.4f}   "
          f"delta {scaled - unscaled:+.4f}")

print("\nDistance-based methods are transformed by scaling. The forest does not")
print("move -- splits depend on order, which scaling preserves.")

```

What just happened. The same three models were cross-validated twice: on the raw columns, and after `StandardScaler` put every column on mean 0, standard deviation 1.

How to read the output. k-NN jumps from about 0.59 to about 0.77, and the SVM from below chance (about 0.48) to about 0.73. The random forest does not move.

Why. k-NN measures the distance between two applicants as $\sqrt{\Delta \text{age}^2 + \Delta \text{income}^2 + \Delta \text{debt}^2 + \Delta \text{pti}^2}$. A difference of one million rupiah in income swamps any possible difference in `pti` (which is around 1), so unscaled k-NN effectively ignores the most useful column. After scaling, a "one standard deviation" difference counts the same in every column. A tree asks questions like "is `pti` > 1.3?"; rescaling changes the number in the question, not which applicants go left or right.

Why it matters. Scale before k-NN, SVM, k-means, PCA, penalised regression and neural networks. Do not bother for trees and forests.

Which scaler? It depends on the outliers (short demo)

199 ordinary values around 50, plus **one** genuine extreme value of 260. We look at where each scaler puts the 199 ordinary points.

```

from sklearn.preprocessing import MinMaxScaler, RobustScaler, MaxAbsScaler

rng_s = np.random.default_rng(17)
x_out = np.concatenate([rng_s.normal(50, 8, 199), [260.0]]).reshape(-1, 1)

rows = []
for name, scaler in [("standard", StandardScaler()), ("min-max", MinMaxScaler()),
                    ("robust", RobustScaler()), ("max-abs", MaxAbsScaler())]:
    z = scaler.fit_transform(x_out).ravel()
    bulk = z[:-1] # the 199 ordinary points
    rows.append({"scaler": name, "bulk_min": bulk.min(), "bulk_max": bulk.max(),
                "bulk_IQR": np.percentile(bulk, 75) - np.percentile(bulk, 25),
                "outlier": z[-1]})
print(pd.DataFrame(rows).round(3).to_string(index=False))

```

What just happened. Each scaler was fitted to the same 200 values. The table shows the range and spread (IQR) of the 199 ordinary points after scaling, and where the single outlier ended up.

How to read the output. **Min-max** sends the outlier to exactly 1 and squashes all 199 ordinary points into roughly 0 to 0.19 — they become nearly indistinguishable. **Max-abs** does the same. **Standard** scaling is better but the outlier still inflates the standard deviation. **Robust** scaling uses the median and IQR, which one outlier cannot move, so the bulk keeps an IQR of about 1 and the outlier simply sits far away (about 19).

Why it matters. Scaling never changes a distribution's *shape*, only its location and spread. When genuine outliers exist, use `RobustScaler`; use `MaxAbsScaler` only for sparse data, because it does not shift zeros.

Letting a straight line bend: binning and splines (short demo)

A linear model fits a straight line. If the truth is a curve, we can change the *features* instead of the model: cut `x` into bins, or expand it into a smooth **spline** basis. The model is still linear — in the new columns.

```

from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import KBinsDiscretizer, SplineTransformer

rng_n = np.random.default_rng(37)
x_n1 = np.sort(rng_n.uniform(0, 10, 260))
y_n1 = np.sin(x_n1 * 0.9) * 2.4 + 0.35 * x_n1 + rng_n.normal(0, 0.55, 260)
grid = np.linspace(0, 10, 240)[:, None]

fits = {
    "straight line": LinearRegression(),
    "8 quantile bins": make_pipeline(KBinsDiscretizer(n_bins=8, encode="onehot-dense",
                                                    strategy="quantile"),
                                   LinearRegression()),
    "cubic spline": make_pipeline(SplineTransformer(n_knots=8, degree=3),
                                LinearRegression()),
}

fig, ax = plt.subplots(figsize=(9, 3.4))
ax.scatter(x_n1, y_n1, s=8, color="grey", alpha=0.5)
for name, est in fits.items():
    est.fit(x_n1[:, None], y_n1)
    r2 = est.score(x_n1[:, None], y_n1)
    ax.plot(grid, est.predict(grid), lw=2, label=f"{name} (R² = {r2:.3f})")
    print(f"{name:<16} R² = {r2:.3f}")
ax.set(title="Same linear model, three different bases", xlabel="x", ylabel="y")
ax.legend(frameon=False)
plt.show()

```

What just happened. One linear regression, fed three versions of the same `x`: raw, 8 bin indicators, and 8-knot spline columns.

How to read the output. R^2 (share of variance explained, 1 = perfect) goes from about 0.23 for the straight line, to about 0.84 with bins, to about 0.92 with the spline. In the plot, the binned fit is a staircase — it jumps at every bin edge — while the spline is smooth.

Why it matters. "Linear model" does not mean "straight-line relationship". Change the basis and a linear model fits curves. A third option, `QuantileTransformer`, maps a column onto a uniform or normal distribution by rank; it is robust to any outlier but throws away the spacing between values.

3. Creating features from domain reasoning (DEMO)

This is the part that cannot be automated. We build a per-customer modelling table from a raw transaction log — the standard route from transactional data to a rectangular dataset.

```
tx = pd.read_csv(DATA / "transactions.csv", parse_dates=["ts"])
print(tx.shape)
tx.head()
```

What just happened. We loaded the transaction log: one row per purchase, with the customer, the time, the amount and the category.

How to read the output. 30,000 rows but only 4 columns, and a customer appears many times. A model predicting something *about customers* needs one row per customer — so the next cell summarises the log per customer.

```

asof = tx["ts"].max()
print("feature cutoff (as-of date):", asof)

feats = tx.groupby("customer_id").agg(
    n_tx=("amount", "size"),
    total=("amount", "sum"),
    mean_amt=("amount", "mean"),
    std_amt=("amount", "std"),
    max_amt=("amount", "max"),
    first_ts=("ts", "min"),
    last_ts=("ts", "max"),
    n_categories=("category", "nunique"),
)

# Recency, Frequency, Monetary -- the classic RFM triple
feats["recency_days"] = (asof - feats["last_ts"]).dt.days
feats["tenure_days"] = (asof - feats["first_ts"]).dt.days
feats["tx_per_month"] = feats["n_tx"] / (feats["tenure_days"] / 30).clip(lower=1)

# Dispersion relative to level: a scale-free measure of erratic spending
feats["cv_amt"] = feats["std_amt"] / feats["mean_amt"]

# Category concentration: is this a specialist or a generalist?
share = tx.pivot_table(index="customer_id", columns="category",
    values="amount", aggfunc="sum", fill_value=0)
feats["top_cat_share"] = share.max(axis=1) / share.sum(axis=1)

# Trend: recent behaviour relative to the customer's own baseline
recent = tx[tx["ts"] > asof - pd.Timedelta(days=90)]
feats["recent_vs_overall"] = (recent.groupby("customer_id")["amount"].mean()
    / feats["mean_amt"]).fillna(0)

cols = ["n_tx", "mean_amt", "cv_amt", "recency_days", "tx_per_month",
    "top_cat_share", "recent_vs_overall"]
print(feats[cols].describe().round(3).T.to_string())

```

What just happened. `groupby("customer_id").agg(...)` turned the log into one row per customer, and we derived new columns from the summaries. **RFM** is the classic minimum: **R**ecency (days since last purchase), **F**requency (purchases per month), **M**onetary (how much they spend).

How to read the output. Each row of the table describes one feature across all customers. For example `recency_days` has a median (50%) of about 16 days but a maximum above 200 — most customers are active, a few have gone quiet. `top_cat_share` near 1 means a customer spends almost everything in one category (a specialist).

The as-of date matters. Every feature is measured relative to `asof`, the last date in the log. If you later predict something that happens *after* that date, the features must not use any data from after it — otherwise you leak the future into the past.

Seven features from four raw columns, each motivated by a **hypothesis about behaviour** rather than an automatic rule. Note the two defensive details: `.clip(lower=1)` prevents division by zero for one-day-old customers, and `.fillna(0)` handles customers with no recent transactions — an omission that would otherwise propagate `NaN` into the model.

Cyclical time features

Hour 23 and hour 0 are adjacent, but the integers 23 and 0 are maximally far apart. Encode them on a circle.

```

tx["hour"] = tx["ts"].dt.hour
tx["hour_sin"] = np.sin(2 * np.pi * tx["hour"] / 24)
tx["hour_cos"] = np.cos(2 * np.pi * tx["hour"] / 24)

def circ_dist(h1, h2):
    p1 = np.array([np.sin(2*np.pi*h1/24), np.cos(2*np.pi*h1/24)])
    p2 = np.array([np.sin(2*np.pi*h2/24), np.cos(2*np.pi*h2/24)])
    return np.linalg.norm(p1 - p2)

print(f"raw integer distance    23:00 vs 00:00 = {abs(23-0):>6.3f}")
print(f"raw integer distance    11:00 vs 12:00 = {abs(11-12):>6.3f}    <- wrong!")
print(f"cyclical distance       23:00 vs 00:00 = {circ_dist(23,0):>6.3f}")
print(f"cyclical distance       11:00 vs 12:00 = {circ_dist(11,12):>6.3f}    <- equal")

```

What just happened. Each hour was placed on a clock face: `hour_sin` and `hour_cos` are its x and y coordinates on a circle of radius 1.

How to read the output. As plain integers, 23:00 and 00:00 are 23 apart even though they are one hour apart. On the circle, both one-hour gaps have the same distance (about 0.26), exactly as they should.

Why two columns? One angle alone still jumps from 360° back to 0° at midnight. The (sin, cos) pair gives every hour a unique point, and neighbouring hours are always neighbouring points. The same trick works for day of week (divide by 7) and month (divide by 12).

TASK 5.1

How many customers are in your feature table?

```

n_pelanggan = None

checks.task("5.1", n_pelanggan)

```

Hint. Each row of `feats` is one customer. If the check fails, ask yourself which table you are counting: `tx` has one row per *transaction*.

TASK 5.2

Build the three credit ratios that the risk model actually depends on: `dti` (debt / income), `pti` (monthly_payment / monthly income) and `util` (balance / credit_limit). Guard every denominator.

```

credit_feat = credit.copy()

# TODO: add the three ratio columns, e.g.
# credit_feat["dti"] = credit_feat["debt"] / credit_feat["income"].clip(lower=1)
# credit_feat["pti"] = ...
# credit_feat["util"] = ...

checks.task("5.2", credit_feat)

```

Hints. Monthly income is `income / 12`. "Guard the denominator" means `.clip(lower=1)` on whatever you divide by, so a zero never produces `inf`. Why these three? Each is a number a credit analyst actually uses: how large the debt is relative to income (`dti`), how much of each month's income the instalment eats (`pti`), and how much of the credit limit is used (`util`).

4. Feature selection (DEMO)

Three families, in increasing order of cost and of fidelity to the final model:

- **Filter** — score each feature against the target on its own, with no model. Fast, but blind to features that only help in combination.
- **Wrapper** — try subsets of features by actually training the model. Accurate, but slow.
- **Embedded** — the model selects while it fits (e.g. the lasso).

```
from sklearn.feature_selection import (mutual_info_classif, f_classif,
                                      SelectKBest, RFECV)
from sklearn.impute import SimpleImputer

num_cols = credit.select_dtypes("number").columns.tolist()
X_num = pd.DataFrame(
    SimpleImputer(strategy="median").fit_transform(credit[num_cols]),
    columns=num_cols)

# Add the ratios, so selection can see them
X_num["dti"] = X_num["debt"] / X_num["income"].clip(lower=1)
X_num["pti"] = X_num["monthly_payment"] / (X_num["income"] / 12).clip(lower=1)
X_num["util"] = X_num["balance"] / X_num["credit_limit"].clip(lower=1)

# --- filter: fast, model-agnostic, blind to interactions -----
f_scores = pd.Series(f_classif(X_num, y)[0], index=X_num.columns)
mi_scores = pd.Series(mutual_info_classif(X_num, y, random_state=0),
                      index=X_num.columns)

comparison = pd.DataFrame({"anova_F": f_scores, "mutual_info": mi_scores})
comparison["rank_F"] = comparison["anova_F"].rank(ascending=False).astype(int)
comparison["rank_MI"] = comparison["mutual_info"].rank(ascending=False).astype(int)
print(comparison.sort_values("mutual_info", ascending=False).round(4).to_string())
```

What just happened. Two **filter** scores for every numeric column (plus our three ratios): the ANOVA **F** statistic (do defaulters and non-defaulters have different *averages* of this feature?) and **mutual information** (does knowing this feature tell you *anything* about default, in any shape?).

How to read the output. Bigger = more related to default. Both methods put `pti`, `term_months` and `monthly_payment` at the top. Several columns get MI of exactly 0 — that means the estimator could not detect a relationship, not that there definitely is none.

The blind spot. Each feature is scored alone. `debt` and `income` look weak individually even though their *ratio* is strong — a filter cannot see that.

Mutual information is the more useful filter because it detects **any** dependence, not only monotone ones. A feature with correlation near 0 and high mutual information — a U-shaped relationship — is invisible to a correlation filter and obvious to this one. The next cell shows exactly that.

```

from sklearn.feature_selection import mutual_info_regression

rng_m = np.random.default_rng(23)
x_mi = rng_m.uniform(-3, 3, 400)
y_u = x_mi ** 2 + rng_m.normal(0, 0.8, 400)          # U-shaped
y_lin = 1.4 * x_mi + rng_m.normal(0, 1.6, 400)        # straight line

fig, axes = plt.subplots(1, 2, figsize=(9, 3.2), sharex=True)
for ax, yy, title in [(axes[0], y_u, "U-shaped"), (axes[1], y_lin, "linear")]:
    r = np.corrcoef(x_mi, yy)[0, 1]
    mi = mutual_info_regression(x_mi[:, None], yy, random_state=0)[0]
    ax.scatter(x_mi, yy, s=6, alpha=0.5, color="#2E6F9E")
    ax.set(title=f"{title}: corr = {r:.3f}, MI = {mi:.3f}", xlabel="x", ylabel="y")
    print(f"{title:<9} correlation = {r:6.3f}    mutual information = {mi:.3f}")
plt.show()

```

What just happened. Two made-up relationships: $y = x^2$ plus noise (a U), and a straight line plus noise. We measured each with correlation and with mutual information.

How to read the output. For the U-shape, correlation is about 0.05 — essentially zero — yet MI is about 1.16, the *highest* of the two. Look at the left plot: x clearly determines y . Correlation only measures *straight line* association; the rising and falling halves of the U cancel out.

Why it matters. A correlation filter would throw this feature away. Mutual information keeps it.

Wrapper: recursive feature elimination

RFECV fits the model, drops the weakest feature, refits, and repeats — scoring every subset size by cross-validation so the *number* of features is chosen by the data, not guessed.

```

Xs_num = StandardScaler().fit_transform(X_num)
rfe = RFECV(LogisticRegression(max_iter=2000), step=1, cv=cv,
             scoring="roc_auc").fit(Xs_num, y)

curve = pd.Series(rfe.cv_results_["mean_test_score"],
                  index=range(1, X_num.shape[1] + 1), name="CV ROC-AUC")
print("CV ROC-AUC by number of features kept:")
print(curve.round(4).to_string())
print(f"\nRFECV keeps {rfe.n_features_} of {X_num.shape[1]}:",
      list(X_num.columns[rfe.support_]))
print("dropped:", list(X_num.columns[~rfe.support_]))

```

What just happened. A **wrapper** method: the logistic regression itself judged which features to keep, by being retrained again and again with one fewer feature each time.

How to read the output. The curve rises steeply for the first three features (about 0.76 → 0.80) and then almost flattens: every further feature adds at most a few thousandths. RFECV picks the size with the best score and lists the survivors — all three engineered ratios are among them. When the curve is this flat, a smaller subset costs almost nothing.

Why it matters. Unlike a filter, a wrapper sees features *together*, through the model you will actually use. The cost is time: it trains the model many times. (We scaled all rows here to keep the demo short; for an honest final score, selection belongs inside a pipeline — see §6.)

```
# --- embedded: selection happens as part of fitting -----
from sklearn.linear_model import LassoCV
from sklearn.feature_selection import SelectFromModel

lasso_sel = SelectFromModel(
    LassoCV(cv=5, random_state=0, max_iter=20000)
).fit(StandardScaler().fit_transform(X_num), y)
kept = X_num.columns[lasso_sel.get_support()]
print(f"L1 (lasso) kept {len(kept)} of {X_num.shape[1]} features:")
print(" ", list(kept))
```

What just happened. An **embedded** method. The lasso adds a penalty on the size of the coefficients, which can push weak coefficients to exactly zero; `SelectFromModel` keeps only the non-zero ones.

How to read the output. Here the lasso kept *every* feature. That is not a bug: `LassoCV` chose the penalty strength that predicts best, and on this data that penalty is too weak to zero anything out.

Why it matters. Embedded selection is not automatic. If you want a short list you must ask for a stronger penalty (or a threshold) and accept slightly worse predictions in exchange.

```
# --- permutation importance: measures what you actually care about ---
from sklearn.inspection import permutation_importance
from sklearn.model_selection import train_test_split

Xtr, Xte, ytr, yte = train_test_split(X_num, y, test_size=0.25,
                                     stratify=y, random_state=42)
rf = RandomForestClassifier(n_estimators=200, random_state=0,
                           ).fit(Xtr, ytr)

perm = permutation_importance(rf, Xte, yte, n_repeats=10,
                              random_state=0, scoring="roc_auc")

imp = pd.DataFrame({
    "impurity": pd.Series(rf.feature_importances_, index=X_num.columns),
    "permutation": pd.Series(perm.importances_mean, index=X_num.columns),
}).sort_values("permutation", ascending=False)
print(imp.round(4).to_string())

print("\nImpurity importance is biased towards high-cardinality and continuous")
print("features. Permutation importance measures the drop in the score you")
print("actually report -- prefer it.")
```

What just happened. Two ways to ask a random forest which features it uses. **Impurity** importance counts how much each feature's splits helped during training. **Permutation** importance takes the *test* set, shuffles one column (breaking its link to the target), and measures how much ROC-AUC drops.

How to read the output. `pti` and `dti` lead on both. Lower down the two disagree: some columns have sizeable impurity importance but almost zero permutation importance, and one is slightly *negative* — the model scores a little *better* with it shuffled, so it was adding noise.

Why it matters. Permutation importance measures the thing you care about — performance on unseen data. Prefer it. But it has one trap, next.

The trap: a feature with a twin (short demo)

If two columns carry the same information, shuffling one does little harm — the model reads the other. We add a near-copy of feature A and watch A's importance.

```

from sklearn.ensemble import RandomForestRegressor

rng_t = np.random.default_rng(52)
n_t = 900
X_t = rng_t.normal(size=(n_t, 5))
y_t = 3.0 * X_t[:, 0] + 1.8 * X_t[:, 1] - 1.2 * X_t[:, 2] + rng_t.normal(0, 1.0, n_t)
X_twin = np.column_stack([X_t, X_t[:, 0] + rng_t.normal(0, 0.02, n_t)]) # A' = A + tiny noise

twin_table = {}
for tag, Xa in (("without A'", X_t), ("with A'", X_twin)):
    a_tr, a_te, b_tr, b_te = train_test_split(Xa, y_t, test_size=0.35, random_state=1)
    rf_t = RandomForestRegressor(n_estimators=140, random_state=1).fit(a_tr, b_tr)
    r = permutation_importance(rf_t, a_te, b_te, n_repeats=12, random_state=1)
    twin_table[tag] = pd.Series(r.importances_mean, index=list("ABCDE") + ["A'"][:Xa.shape[1] - 5])
    print(f"{tag:<11} test R^2 = {rf_t.score(a_te, b_te):.3f}")
print()
print(pd.DataFrame(twin_table).round(3).to_string())

```

What just happened. y depends on A (strongly), B and C; D and E are noise. We measured permutation importance twice: without and with A', a copy of A with 2% noise.

How to read the output. The model is equally accurate both times (same R^2). But A's importance falls from about 1.31 to about 0.33 once A' exists, and A' gets a similar share (about 0.37). Each now looks no more important than B, although together they carry most of the signal.

Why it matters. A near-zero permutation importance does **not** prove a feature is useless — first check whether it has a correlated twin. In the credit data, `debt` and `dti` partly overlap in exactly this way.

5. Principal component analysis (DEMO)

PCA looks for new axes — **principal components** — that are combinations of the original columns. The first axis points in the direction where the data is most spread out (greatest variance); the second is the most spread-out direction at right angles to the first; and so on. Keeping the first few gives a smaller dataset that loses as little variance as possible.

Mathematically, the components are the **eigenvectors of the covariance matrix**, ordered by eigenvalue, and component j explains $\frac{\lambda_j}{\sum_l \lambda_l}$ of the total variance.

First, verify that claim numerically.

```

from sklearn.decomposition import PCA
from sklearn.datasets import load_breast_cancer

Xbc, ybc = load_breast_cancer(return_X_y=True, as_frame=True)
Xs = StandardScaler().fit_transform(Xbc)

# Route 1: eigendecomposition of the covariance matrix, by hand
cov = np.cov(Xs, rowvar=False)
eigvals, eigvecs = np.linalg.eigh(cov)
eigvals = eigvals[::-1] # eigh returns ascending

# Route 2: scikit-learn's PCA (which uses the SVD)
pca = PCA().fit(Xs)

print("first 5 eigenvalues, by hand :", np.round(eigvals[:5], 4))
print("first 5 from sklearn      :", np.round(pca.explained_variance_[:5], 4))
print("identical:", np.allclose(eigvals[:5], pca.explained_variance_[:5]))

```

What just happened. The breast-cancer dataset: 569 tumours, 30 measurements each, standardised. We computed PCA two ways — by hand, as the eigenvalues of the covariance matrix, and with scikit-learn.

How to read the output. The two lists are identical. Each eigenvalue is the variance of the data *along* that component: the first component alone has a variance of about 13.3, whereas each original standardised column has variance 1.

Why it matters. PCA is not a black box. It is this eigendecomposition, and `explained_variance_` is just the list of eigenvalues.

```
evr = pca.explained_variance_ratio_
cum = np.cumsum(evr)

print(f"original dimensions: {Xbc.shape[1]}")
for k in (1, 2, 3, 5, 10):
    print(f"  {k:>2} components: {cum[k-1]:.3%} of variance")
print(f"\ncomponents for 95% variance: {np.searchsorted(cum, 0.95) + 1}")
print(f"eigenvalues > 1 (Kaiser)    : {(pca.explained_variance_ > 1).sum()}")
print("\nThe two criteria disagree. Neither is wrong; they answer different questions.")
```

What just happened. Each eigenvalue divided by their total is that component's share of the variance. Adding the shares up gives the **cumulative** variance kept by the first k components.

How to read the output. One component keeps about 44% of the information in 30 columns; ten keep about 95%. Two rules for choosing k disagree: "keep 95%" says 10, while Kaiser ("keep components whose eigenvalue is above 1, i.e. worth more than one original column") says 6.

Why it matters. There is no single right k . If PCA feeds a model, treat k as a hyperparameter and let cross-validation choose it.

```
fig, axes = plt.subplots(1, 2, figsize=(11, 3.4))
axes[0].bar(range(1, 16), evr[:15], color="#2E6F9E")
axes[0].axhline(1 / len(evr), color="grey", ls=":", lw=1)
axes[0].set(title="Scree plot", xlabel="component", ylabel="variance explained")
axes[1].plot(range(1, len(cum) + 1), cum, marker="o", ms=3, color="#173F5F")
axes[1].axhline(0.95, color="#9C3535", ls="--", lw=1)
axes[1].set(title="Cumulative variance", xlabel="components",
            ylabel="cumulative proportion")
plt.show()
```

How to read the figure. Left, the **scree plot**: one bar per component, tallest first. Look for the "elbow" where the bars stop dropping sharply and flatten into a tail — here after about 2–3 components. The dotted line is what each component would get if variance were spread evenly ($1/30$). Right, the cumulative curve crossing the dashed 95% line at component 10.

Why it matters. That is a third rule for choosing k — the elbow — and it gives yet another answer. The 30 measurements are highly redundant: a few components carry most of the information.

```
# Loadings are the only route to interpreting what a component MEANS.
loadings = pd.Series(pca.components_[0], index=Xbc.columns)
print("PC1, largest loadings:")
print(loadings.abs().nlargest(6).round(3).to_string())
print("\nPC1 is dominated by concavity and concave-point measures -- it is")
print("essentially a 'boundary irregularity' factor.")
```

What just happened. PC1 is a weighted sum of all 30 original columns. The weights are called **loadings**; we printed the six largest.

How to read the output. The top loadings are all about concavity and concave points — how dented and irregular the tumour's outline is. The weights are similar in size (about 0.23–0.26), so PC1 is a broad average of these measurements, not one column in disguise.

Why it matters. Reading the loadings is the *only* way to say what a component means. Here PC1 happens to match what doctors look for in malignant tumours — a lucky property of this dataset, not a guarantee.

Four limitations, frequently ignored:

- PCA is **unsupervised**. It maximises variance in X with no reference to y ; a low-variance component may carry all your signal.
- PCA **requires scaling**. Without it, whichever variable has the largest units determines the components.
- PCA is **linear**. Curved manifolds need kernel PCA, t-SNE or UMAP — and the latter two are visualisation tools, not stable transforms for new data.
- PCA **destroys interpretability**. Component 1 is a weighted combination of all 30 original variables.

```
# Demonstrating limitation 2: PCA without standardising.
pca_unscaled = PCA().fit(Xbc.to_numpy())
cum_unscaled = np.cumsum(pca_unscaled.explained_variance_ratio_)
print(f"components for 95%, standardised : {np.searchsorted(cum, 0.95) + 1}")
print(f"components for 95%, NOT standardised: {np.searchsorted(cum_unscaled, 0.95) + 1}")
top_unscaled = pd.Series(pca_unscaled.components_[0],
                        index=Xbc.columns).abs().nlargest(3)
print("\nUnscaled PC1 is essentially one variable:")
print(top_unscaled.round(4).to_string())
print("\nThat is an apparently excellent reduction which has discarded")
print("everything not correlated with tumour area.")
```

What just happened. The same PCA, but on the raw measurements instead of standardised ones.

How to read the output. Now a *single* component reaches 95% — which looks like a spectacular result. But its loadings are almost entirely **worst area** and **mean area**. Areas run into the thousands, while concavity is a fraction between 0 and 1, so the area columns have by far the largest variance and PCA simply follows them.

Why it matters. Unscaled, PCA answers "which column has the biggest numbers?" — a fact about units, not about tumours. Standardise before PCA. And be suspicious whenever a result looks *too* good.

6. Leakage — the most important section in this course (DEMO)

Data leakage is the use, during training, of information unavailable at prediction time. It produces validation scores that are optimistic and cannot be reproduced in production.

Here is its cost, measured on data where the target is **pure noise** and no feature has any predictive value whatsoever.

```

from sklearn.pipeline import Pipeline

rng2 = np.random.default_rng(0)
n_rows, p_cols = 200, 5000
X_noise = rng2.normal(size=(n_rows, p_cols))      # pure noise
y_noise = rng2.integers(0, 2, n_rows)              # unrelated: true AUC is 0.5
cv_leak = StratifiedKFold(5, shuffle=True, random_state=1)

# WRONG: select features on ALL the data, then cross-validate the classifier
X_sel = SelectKBest(f_classif, k=20).fit_transform(X_noise, y_noise)
leaky = cross_val_score(LogisticRegression(max_iter=1000), X_sel, y_noise,
                        cv=cv_leak, scoring="roc_auc").mean()

# RIGHT: selection inside the pipeline, refitted on each training fold
pipe = Pipeline([("sel", SelectKBest(f_classif, k=20)),
                  ("clf", LogisticRegression(max_iter=1000))])
honest = cross_val_score(pipe, X_noise, y_noise, cv=cv_leak,
                        scoring="roc_auc").mean()

print(f"leaky selection : ROC-AUC = {leaky:.3f}")
print(f"inside pipeline : ROC-AUC = {honest:.3f}")
print(f"truth           : ROC-AUC = 0.500")

```

There is **no signal in this data at all**. The leaky procedure reports 0.91, which would be an excellent model if it were real.

The mechanism is simple: with 5,000 noise features and 200 rows, some features correlate with the target by chance; the selector finds exactly those; and cross-validation then measures a model built using the very rows it is being validated against.

Step by step, in the WRONG version:

1. `SelectKBest` looks at **all 200 rows and their labels** and keeps the 20 columns that happen to line up best with the coin flips.
2. Cross-validation then splits the rows into train and test folds.
3. But each test fold's labels *already* helped choose the 20 columns. The test fold is no longer unseen — it has leaked into the model.

In the RIGHT version the selector is a step *inside* the pipeline, so within each fold it is fitted on the training rows only. The test fold is genuinely unseen, and the score falls to near 0.5, the truth. (It is not exactly 0.5 because 200 rows is a small, noisy sample.)

This is not a contrived edge case. **Any** workflow with more features than rows — genomics, text, sensor arrays, or a spreadsheet after aggressive feature generation — is exposed to it.

```
# How the illusion scales with the number of noise features.
# Each p gets a fresh draw from the same seed, as in the lecture slides.
results = []
for p in (50, 200, 500, 1500, 5000):
    rng_p = np.random.default_rng(0)
    Xn = rng_p.normal(size=(n_rows, p))
    yn = rng_p.integers(0, 2, n_rows)
    lk = cross_val_score(LogisticRegression(max_iter=1000),
                        SelectKBest(f_classif, k=20).fit_transform(Xn, yn),
                        yn, cv=cv_leak, scoring="roc_auc").mean()

    hn = cross_val_score(
        Pipeline([("sel", SelectKBest(f_classif, k=20)),
                  ("clf", LogisticRegression(max_iter=1000))]),
        Xn, yn, cv=cv_leak, scoring="roc_auc").mean()
    results.append({"n_noise_features": p, "leaky_AUC": lk, "honest_AUC": hn})

leak_table = pd.DataFrame(results)
print(leak_table.round(3).to_string(index=False))
print("\nThe more features you have, the bigger the lie.")
```

What just happened. The same leaky-versus-honest comparison, repeated with 50, 200, 500, 1,500 and 5,000 noise columns (always 200 rows, always choosing 20).

How to read the output. The **leaky** column climbs steadily as **p** grows. The **honest** column wobbles around 0.5 the whole time, as it must — there is never any signal. These are the same numbers as the lecture slide.

Why it matters. More candidate columns = more chances for some to match the labels by luck = a bigger fake score. The lie grows with the number of features you search through, not with the quality of your data.

TASK 5.3

Report the ROC-AUC from the **leaky** procedure at $p = 5000$.

```
auc_bocor = None

checks.task("5.3", auc_bocor)
```

Hint. You do not need new code — the value is already stored in a variable from the first cell of this section.

TASK 5.4

Report the ROC-AUC from the **honest** pipeline at $p = 5000$.

```
auc_jujur = None

checks.task("5.4", auc_jujur)
```

Think about it. Subtract your two answers. That gap is how much score a single misplaced line of code can invent — on data with *nothing* in it.

7. The leak-proof pipeline (DEMO)

`Pipeline` and `ColumnTransformer` make the correct version the *easy* one: every fitted quantity — the imputation median, the scaler's mean and standard deviation, the encoder's categories, the selected features — is learned on the training fold alone, automatically, including inside cross-validation.

```
from sklearn.compose import ColumnTransformer
from sklearn.base import BaseEstimator, TransformerMixin

class CreditFeatures(BaseEstimator, TransformerMixin):
    """Domain ratios, computed INSIDE the pipeline so they cannot drift
    between training and serving."""

    def fit(self, X, y=None):
        return self

    def transform(self, X):
        X = X.copy()
        X["debt_to_income"] = X["debt"] / X["income"].clip(lower=1)
        X["loan_to_income"] = X["loan_amount"] / X["income"].clip(lower=1)
        X["payment_to_income"] = X["monthly_payment"] / (X["income"] / 12).clip(lower=1)
        X["credit_utilisation"] = X["balance"] / X["credit_limit"].clip(lower=1)
        X["income_per_dependent"] = X["income"] / (1 + X["dependents"])
        X["age_at_maturity"] = X["age"] + X["term_months"] / 12
        return X

numeric = ["age", "income", "debt", "loan_amount", "term_months", "balance",
           "credit_limit", "dependents", "monthly_payment",
           "debt_to_income", "loan_to_income", "payment_to_income",
           "credit_utilisation", "income_per_dependent", "age_at_maturity"]
nominal = ["purpose", "housing", "employment_type"]
ordinal = ["education"]
edu_order = [["SMA", "D3", "S1", "S2"]]

preprocess = ColumnTransformer([
    ("num", Pipeline([("imp", SimpleImputer(strategy="median", add_indicator=True)),
                     ("sc", StandardScaler())]), numeric),
    ("nom", Pipeline([("imp", SimpleImputer(strategy="constant",
                                             fill_value="Unknown")),
                     ("oh", OneHotEncoder(handle_unknown="infrequent_if_exist",
                                             min_frequency=0.01,
                                             sparse_output=False))]), nominal),
    ("ord", Pipeline([("imp", SimpleImputer(strategy="most_frequent")),
                     ("oe", OrdinalEncoder(categories=edu_order,
                                             handle_unknown="use_encoded_value",
                                             unknown_value=-1)),
                     ("sc", StandardScaler())]), ordinal),
], remainder="drop", verbose_feature_names_out=False)

full = Pipeline([
    ("features", CreditFeatures()),
    ("prep", preprocess),
    ("clf", LogisticRegression(max_iter=3000, C=0.5, class_weight="balanced")),
])
```

What just happened. Nothing ran yet — this cell only *describes* the whole process as one object, `full`. Read it top to bottom:

1. `CreditFeatures` adds six domain ratios.
2. `ColumnTransformer` sends each column type down its own route: numeric → fill gaps with the median (and add a "was missing" flag) → standardise; nominal → fill with "Unknown" → one-hot; ordinal (`education`) → fill → encode in the stated order → standardise.
3. `LogisticRegression` makes the prediction.

Why it matters. Because everything is one object, `full.fit(...)` learns every median, mean and category from the rows it is given — and nothing else. Cross-validation calls `fit` on each training fold separately, so no test fold can leak in.

```
# Split FIRST. Everything else happens inside the pipeline.
X_all = credit.drop(columns=["default"])
Xtr2, Xte2, ytr2, yte2 = train_test_split(X_all, y, test_size=0.2,
                                         stratify=y, random_state=42)

scores = cross_val_score(full, Xtr2, ytr2, cv=cv, scoring="roc_auc")
print(f"CV ROC-AUC (train only): {scores.mean():.4f} +/- {scores.std():.4f}")

full.fit(Xtr2, ytr2)
from sklearn.metrics import roc_auc_score
held_out = roc_auc_score(yte2, full.predict_proba(Xte2)[:, 1])
print(f"held-out ROC-AUC      : {held_out:.4f}")
print("\nThe two agree, which is what an honest pipeline looks like.")
```

What just happened. We held back 20% of the applications *before doing anything else*. Cross-validation ran on the other 80%; then the pipeline was fitted once on that 80% and scored on the untouched 20%.

How to read the output. The cross-validated score (about 0.83) and the held-out score (about 0.82) are close. The small gap is ordinary sampling noise. A leaky pipeline shows the opposite pattern: a CV score far above the held-out score.

Why it matters. Printing both numbers side by side is the cheapest leak check there is. Compare with §2, where four columns reached at most about 0.77: more engineered ratios plus honest preprocessing make the difference.

```
# Did the engineered ratios earn their place?
names = full.named_steps["prep"].get_feature_names_out()
coefs = pd.Series(full.named_steps["clf"].coef_[0], index=names)
print("Largest |coefficient| (standardised, so comparable):")
print(coefs.abs().nlargest(10).round(3).to_string())
```

What just happened. We pulled the fitted logistic-regression coefficients out of the pipeline and listed the ten largest in size.

How to read the output. Because every input was standardised, a bigger coefficient means a bigger effect on the predicted risk per standard deviation. `debt_to_income` tops the list, and several other engineered ratios are in the top ten — columns that did not exist in the CSV. `employment_type_Unknown` is the *absence* of an answer, and it predicts default: missingness itself can be information.

Careful. A large coefficient means the feature helps the model *rank* risk. It does not mean changing that feature would change a person's risk.

8. Take-home (assessed)

A. Encoding trade-offs. A categorical feature has 3,000 levels, of which the top 20 cover 85% of rows. Compare one-hot, target and hashing encodings: state for each the resulting dimensionality, the principal risk, and the circumstance in which you would choose it.

B. Leakage at scale. Reproduce section 6, then vary p over $\{50, 500, 5000\}$ and **plot** leaky and honest ROC-AUC against p . Explain the shape of each curve.

C. PCA without scaling. Reproduce the breast-cancer PCA *without* standardising. Report how many components now reach 95%, examine PC1's loadings, and explain mechanically what happened.

D. Three more features. Add three features of your own design to the RFM table of section 3: one capturing weekday-vs-weekend behaviour, one capturing spending volatility over time, and one capturing category switching. State the behavioural hypothesis behind each, then test whether they improve a model.

E. Audit a leak. A colleague's churn model scores ROC-AUC 0.98 in cross-validation and 0.61 in production. Their features include `days_since_last_login`, `support_tickets_open`, `account_status` and `final_invoice_amount`. Identify the likely culprit, classify the **type** of leakage, and describe the audit you would run over the remaining features.

Next week: our first models.